AIML402: Assignment 1 report

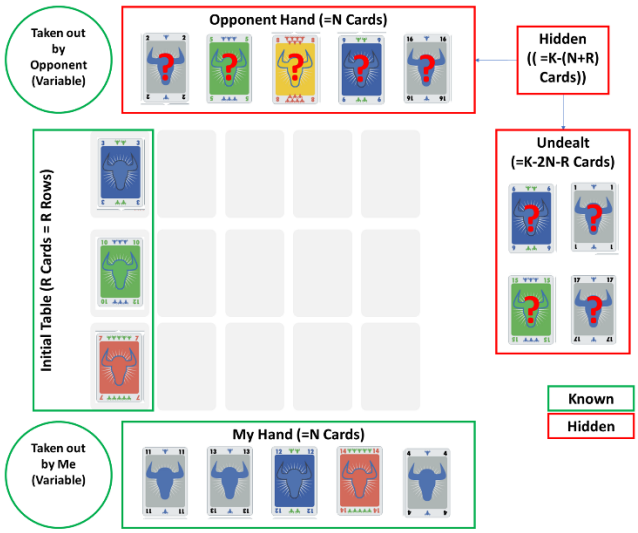
Siddharth Barjatya ( )

August 12th, 2025

This document is a submission for Assignment 1 : Task 2 - a report explaining the strategy of your agent and providing some results of its evaluation.

Brief Introduction

The game of XNimmt is an multi-agent adversarial search problem, where the agents make simultaneous play, although due to the game design the cards are resolved one after the other. This mimics a multi-agent scenario where we have to model the consequences of our own action agent against the opponent agent's moves and reactions. The game of XNimmt is not designed to be an alternating two-player game, like chess, where players take turns to make a move. It is expected that both players will make simultaneous moves without having any prior knowledge of what the opponent choice would be. However it doesn't strictly fit the zero-sum game category as well, where anything which is good for one player is bad for the other player. In case of XNimmt, both players can make a move with no bad consequences for either, at least in that turn. Both players/agents have imperfect information and a lot depends on chance as well. Even though no new cards are dealt during the play, once a new hand is dealt to both players, there is still lack of perfect information about undealt or opponent's cards.



This report outlines the mechanics a minimax-like search-based agent that plays a the game of Nimmt!. The game consists of a deck of K cards, a table of R rows, and a starting hand of N cards. The objective of the game is to take the least number of points. Number of bulls on the taken cards are considered points and a sum is calculated at the end of N card play. The player who has the least sum of points, wins!

- The initial state (s0) in this game could be defined as the state each row is placed with a card from the deck. So initial state has R numbers of cards placed i.e. one in each row.
- A set of possible actions (legal moves), {a1, a2, ...} when in state s in this game is to choose any card from the cards left in the agent's hand. All card selections are considered legal as there is no restriction on what an agent can or cannot play. An agent can also choose to play the cards from the hand at random.
- A transition model, which returns the state s that results from taking action/move a when in state s (i.e. s = TM(s, a) is provided by the function table_resolve() that returns updated table state.
- The utility function – agent's subjective value of a terminal state is the sum of bulls scored as a result of the cards taken during the course of the game.
- The game is considered to be in terminal state when all cards from the hand are placed on the table and final evaluation is done. Either of the players scored less is considered the winner.

How the agent works – the strategy, high level algorithm

My agent function deploys a min-max algorithm that recursively evaluates the game states and returns the best card to play. It uses alpha-beta pruning to optimize the search process to avoid unnecessary evaluations. This function evaluates the game to a max depth. The max depth is a hard coded value in order to limit the search space and improve performance. The main function minimax takes table, my_hand, opp_hand, depth, alpha, beta, maximizing_player(T/F) as inputs and recursively jumps between the MAX and MIN gameplay turns. It keeps updating the game state with every assumed turn and keep a track of an evaluation score for each parent node. Whenever

1. The depth limited minimax searches the tree until depth limit is reached.
2. Applys a heuristic evaluation function to the state of each leaf node to estimate the utility of the corresponding state.
3. Backs up the values of the nodes to the parent, selecting maximum value on
4. Continue backing up until it gets values for the children of the root node.

The table_resolve() provided by the xnimmt.py file function returns a negative value which is assigned to loss as a positive number to avoid confusion. This is to be seen as a penalty to my evaluation score if MAX's move is incurring a loss, whereas if MIN is making a move then its loss gets added to my evaluation score.

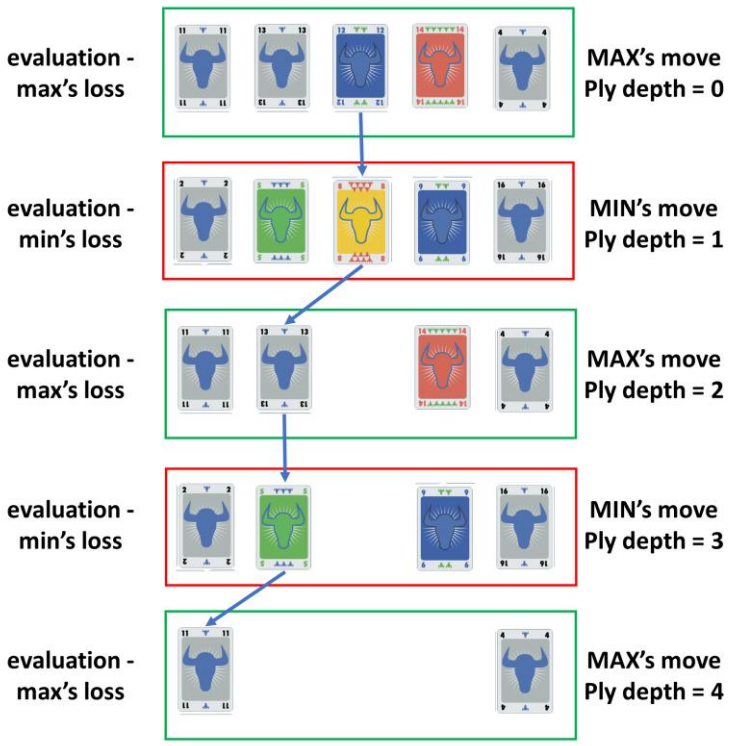
The eval_score variable holds the dynamic value of the evaluation while jumping across the MIN & MAX decisions. The alpha value (i.e. partial evaluation value for MAX node's children is held by alpha variable while partial evaluation value for MIN node's children is held by beta variable. The alpha-beta pruning, evaluates the nodes and back up node values as much as we can while building the search tree. As the tree is searched depth-first, the value a node is backed up to the parent before the siblings are evaluated. The algorithm breaks out of the recursive loop and tries a new node when beta <= alpha.

We want to minimize the loss, so it is subtracted from the evaluation score. Whereas on the other hand, opponent's loss is added to our eval_score variable as opponent's penalty is good for my agent. Note that alpha, beta, eval_score and loss variables are not deep copies so that their memory location remains the same and even though the function moves between MAX and MIN, the eval_score keeps getting updated.

Note : *My mimimax search based agent doesn't strictly follow the gameplay provided in the assignment as it assumes each player to take turns to play. Since it was challenging to add both the recursive search algorithm with unknown opponent hand, it closely mimics the gameplay while making a key assumption that it can go first. Since the algorithm considers taking turns rather than considering both cards to be placed at once, the search tree evaluates its own move at Ply depth 0 and the opponent's move at Ply depth 1. Although there might seem an inherent flaw in this logic, however as the updated table is provided considering our agent to move first, the loss calculated follows the provided logic in the table_resolve() function.*

This is due to the fact that

- Phase 1 All players select one card from their hand to play; each player makes the selection in secret (from the others) and all the selected to play cards are revealed at the same time
- Phase 2 In the ascending order of the number of the selected/revealed cards, each is added to the table



Gameplay Heuristics & Strategies

Strategy : Improving percepts by tracking unknown cards (hidden_cards variable)

Hidden cards are tracked with a helper function called track_cards that takes table and my_hand as parameters and returns the hidden cards. The functions remembers cards that were seen previously seen on the table and/or taken out. It resets the memory of the stored cards every time a new hand is dealt (i.e., when the number of cards in hand is equal to max_cards_in_hand). Keeping track of previously seen and hidden cards helps in dealing a

random hand for the opponent, which is necessary for the minimax algorithm to work efficiently as it helps in limiting the search space.

Strategy : Reducing the role of chance by trying multiple opponent hands (max_width variable)

This is achieved by iterating over the hidden cards in order to generate multiple random hands for the opponent to simulate the inherent uncertainty in the opponent's hand. Each unique instance is then run through the minimax algorithm to collect the good card choices. The best card is the one that appears most frequently in the good cards. This is a simple heuritic to choose the most promising card across any instance of opponent's hand. Since a random opponent hand is equally likely, the most common card is the best choice.

Strategy : Lowering the sum of bull values left in hand as a heuristic

As we are aware that at the end of the game the bulls accumulated in the taken pile is considered as the negative score, this heuristic is used to inform the minimax algorithm that a lower total card values is better such that smaller sums give higher evaluation scores. This usually works as it was observed that sometimes the last high value card in hand is played automatically and resolves to a high loss. The heuristic also helps avoid such misadventures.

Analysis of the agent's performance

The agent works perfectly well under various game conditions, and all functions have been coded to be dynamic in nature drawing values from configurable parameters: P number players, K-card deck, R rows on the table, N cards in the starting hand, and Xth card taking a row. Below table shows results from 28 unique tests for the below game settings :

- K: 17 (i.e. total number of cards in the deck)
- N: 5 (i.e. number of cards each player has at the start of the game)
- XthCardTakes: 4 (i.e. the card that takes the row)
- R: 3 (i.e. number of rows on the table)
- totalNumberOfGames: 100, # total number of games played

max_depth	max_width	my_agent	random	Total running time	Score X	Status
8	10	-4.76	-6.54	5 min, 18 s	1.37	Time Out
7	10	-4.05	-6.89	2 min, 21 s	1.70	Won
7	10	-3.66	-7.49	3 min, 04 s	2.05	Won
7	10	-3.85	-7.23	3 min, 06 s	1.88	Won
6	10	-4.67	-6.2	1 min, 20 s	1.33	Won
6	10	-5.13	-6.71	1 min, 40 s	1.31	Won
6	10	-4.87	-6.22	1 min, 43 s	1.28	Won
5	10	-4.98	-7.6	0 min, 39 s	1.53	Won
5	10	-6.03	-5.65	0 min, 38 s	0.94	Lost
5	10	-4.82	-7.2	0 min, 39 s	1.49	Won
8	5	-4.81	-6.93	2 min, 25 s	1.44	Won
8	5	-5.03	-6	2 min, 00 s	1.19	Won

8	5	-4.69	-6.11	2 min, 44 s	1.30	Won
7	**5**	**-4.68**	**-6.77**	**1 min, 28 s**	**1.45**	**Won**
7	**5**	**-4.56**	**-7.12**	**1 min, 35 s**	**1.56**	**Won**
7	**5**	**-4.53**	**-7.05**	**1 min, 11 s**	**1.56**	**Won**
6	5	-5.02	-6.43	0 min, 57 s	1.28	Won
6	5	-4.58	-6.33	0 min, 40 s	1.38	Won
6	5	-5.51	-6.75	0 min, 49 s	1.23	Won
8	1	-4.36	-6.77	0 min, 26 s	1.55	Won
8	1	-4.46	-6.51	0 min, 25 s	1.46	Won
8	1	-3.79	-7.9	0 min, 32 s	2.08	Won
7	1	-4.85	-6.71	0 min, 19 s	1.38	Won
7	1	-5.2	-6.44	0 min, 16 s	1.24	Won
7	1	-4.79	-6.33	0 min, 20 s	1.32	Won
6	1	-4.42	-7.88	0 min, 11 s	1.78	Won
6	1	-4.64	-7.46	0 min, 09 s	1.61	Won
6	1	-5.76	-6.7	0 min, 09 s	1.16	Won

The solutions where 10 randomized possibilities of the opposition hands are considered beats the random agent by even close to 2x score, however this goes beyond the stipulated 5 mins runtime and hence is not considered an optimal solution. On the other hand, when reducing search tree depth to 5, we see that my agent stops winning against the random agent, and hence is also not feasible.

Hence the solutions lies between depth>5 and width<10. From the above table it can be inferred that using a combination of search depth of 7 along with width of random opponent hands as 5 is ideal for reaching an efficient as well as effective solution. This combination plays 100 games under 2 mins and have a ratio of score of around 1.5x. That means random agent consistently scores 1.5x times worse score than my agent on average across 100 games each.

Conclusion – summary/discussion of the results

In conclusion, its demonstrated that a depth-limited minimax algorithm enhanced with alpha–beta pruning provides a robust and effective approach for the game of XNimmt. By incorporating heuristics for tracking hidden cards, simulating multiple opponent hands, and minimizing residual bull values, the agent consistently outperforms a random baseline.

Evaluation across 28 unique configurations shows that performance depends heavily on balancing search depth and breadth. Excessive breadth (e.g., max_width = 10) yields higher win margins but exceeds the 5-minute runtime constraint, while shallow depth (max_depth ≤ 5) leads to inconsistent victories.

The optimal configuration for the K=17, N=5 game parameters is a search depth of 7 and opponent hand width of 5. This combination achieves an average score ratio of 1.5× better than the random agent, while completing 100 games in under two minutes—satisfying both performance and efficiency requirements.